

OAT 1 – Mecânica BELÉM

Autor: Érica Araujo

Mickael Cedraz Alencar

Monyc Luisa Almeida de Cerqueira

Nalbert de Souza Santana

Pedro César Paixão de Jesus

Agenda

O objetivo dessa apresentação é mostrar como os dados foram preparados e validados para análise temporal.

1. Item 1:
Contexto e Problema

2. Item:
Tratamento e Índice

3. Item:
Inspeção dos Dados

4. Item:
Validação e Auditoria

5. Item:
Conclusão

Contexto da atividade



- Trabalhamos com o histórico de atendimentos e serviços da MecâniQA.
- Por conter registros ordenados no tempo, tratamos a base como uma série temporal.
- Nosso foco foi preparar e estruturar os dados antes de aplicar qualquer modelo preditivo.
- Pergunta central: Como converter a coluna Data para que o Pandas reconheça o eixo temporal adequadamente?

Passos executados no jupyter

- Importar CSV com pandas
- Remover espaços nos nomes das colunas
- Converter Data para datetime
- Definir Data como índice
- Ordenar cronologicamente

```
import pandas as pd

# Importação do dataset
df = pd.read_csv("mecanika_dataset.csv")

print("Dataset importado com sucesso!")
df.head()
```

Dataset importado com sucesso! ...

```
# Remove possíveis espaços dos nomes das colunas
df.columns = df.columns.str.strip()

# Converte a coluna Data para datetime
df["Data"] = pd.to_datetime(
    df["Data"],
    format="%Y-%m-%d",
    errors="raise"
)

# Define Data como índice e organiza cronologicamente
df = df.set_index("Data")
df = df.sort_index()

print("Indexação temporal concluída!")
df.head()
```

Resultado da inspeção inicial



- Após a estruturação do índice temporal, realizamos a verificação da qualidade da informação e detecção de dados ausentes:

```
1): df.info()

<class 'pandas.DataFrame'>
DatetimeIndex: 731 entries, 2024-01-01 to 2025-12-31
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Trocas_Oleo      725 non-null    float64
1   Manutencao_Motor 728 non-null    float64
dtypes: float64(2)
memory usage: 17.1 KB

1): print("Quantidade de registros:", df.shape[0])
print("Quantidade de colunas:", df.shape[1])
print("Atributos principais:", [df.index.name] + df.columns.tolist())
print("Tipo do índice:", type(df.index))

Quantidade de registros: 731
Quantidade de colunas: 2
Atributos principais: ['Data', 'Trocas_Oleo', 'Manutencao_Motor']
Tipo do índice: <class 'pandas.DatetimeIndex'>
```

Análise Estatística

- **Valores ausentes:** 6 em Trocas_Oleo e 3 em Manutencao_Motor
- **Médias diárias:** ~26 trocas de óleo e ~10 manutenções
- **Ponto de atenção:** mínimo de -2 em Manutencao_Motor
- **Picos observados:** 150 em óleo e 85 em motor

```
[6]: print("Valores ausentes por atributo:")  
print(df.isnull().sum())
```

```
Valores ausentes por atributo:  
Trocas_Oleo      6  
Manutencao_Motor  3  
dtype: int64
```

```
[7]: df.describe()
```

```
[7]:
```

	Trocas_Oleo	Manutencao_Motor
count	725.000000	728.000000
mean	26.020690	10.179945
std	11.620655	5.419338
min	0.000000	-2.000000
25%	18.000000	7.000000
50%	26.000000	10.000000
75%	33.000000	13.000000
max	150.000000	85.000000

```
[ ]:
```

Considerações finais



- O índice passou a ser DatetimeIndex.
- A série foi ordenada de 01/01/2024 a 31/12/2025.
- Foram identificados dados ausentes nas duas variáveis.

Referências



- LET'S DATA. ***Séries temporais com Python***. YouTube, 10 set. 2021. O vídeo explica conceitos de séries temporais e aplicações com Python.
- OLIVEIRA, Rogério de; ALBARRACIN, Orlando Yesid Esparza; SILVA, Gustavo Rocha da. ***Introdução às séries temporais: uma abordagem prática em Python***. São Paulo: Editora Mackenzie, 2024.
- ALURA. ***Séries temporais: detectando anomalias e realizando previsões***. Alura.